

# HarnessBank: Semantic Gene-Bank Search with Gated Verification for Agent-Harness Self-Evolution

Xiaotian Luo<sup>1,2</sup>, Dizhan Xue<sup>1,2</sup>, Fengxingyu Wang<sup>1,2</sup>, Chuanrui Hu<sup>1,2</sup>, Yafeng Deng<sup>1,2\*</sup>

<sup>1</sup>EverMind AI <sup>2</sup>Shanda Group

{xiaotian.luo, dizhan.xue, ethan.wang}@evermind.ai, {chuanrui.hu, dengyafeng}@shanda.com

## Abstract

Large Language Models (LLMs) have enabled capable agents across diverse applications. Beyond the foundation model, the performance of an agent is governed by the surrounding *agent harness*, including prompts, tools, control loops, etc. Automatically evolving this harness offers a promising pathway to agent improvement, yet existing approaches typically rely on greedy candidate selection and noisy self-generated feedback, rendering their gains susceptible to search collapse, task-specific overfitting, and poor verifiability. To tackle these challenges, we introduce **HarnessBank**, a trustworthy agent-harness self-evolution framework that pairs a task agent with a separate evolver agent for iterative failure diagnosis, harness generation, and evolution verification. HarnessBank maintains a Harness Gene Bank composed of high-performing harnesses of different semantic coordinates. Those harnesses are reinvented, recombined, screened, and selected during the self-evolution procedure. Moreover, we propose a Gated Harness Screening mechanism to efficiently filter high-quality harnesses and reduce the cost of evaluating numerous offspring harnesses. Across seven agent benchmarks, HarnessBank produces consistent performance improvements from 5.1% to 15.4%. Cross-model experiments further verify that the improvements come from the model-specific self-evolving process, instead of a universally optimal harness. Our code will be publicly available upon acceptance.

## 1 Introduction

Large Language Models (LLMs) have enabled increasingly capable autonomous agents that can reason, use tools, interact with environments, and complete complex tasks ranging from software engineering to web navigation (Yao et al. 2023; Zhang et al. 2024; He et al. 2024). Beyond the foundation model, the empirical performance of an agent is fundamentally governed by the surrounding *agent harness* (Yang et al. 2024; Khattab et al. 2024; Agrawal et al. 2026). The same frozen model can exhibit substantially different performance under different system prompts, injected knowledge, tool interfaces, control loops, recovery mechanisms, and runtime configurations. Unlike model weights, which may be closed, rented, or prohibitively expensive to update, the harness is often the only component that can be modified during deployment. Yet harnesses remain largely hand-engineered, making

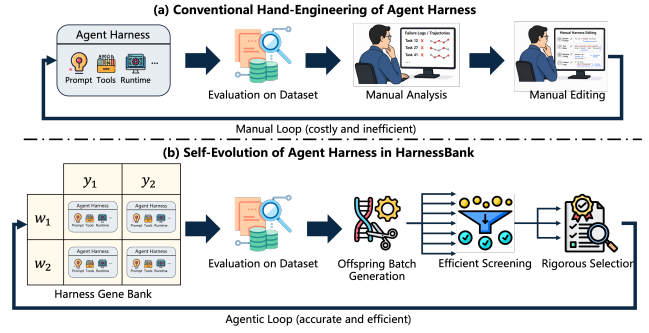


Figure 1: Conventional Hand-Engineering vs. Agent-Harness Self-Evolution: manual loop of agent harness development is costly and inefficient while self-evolution of agent harness in HarnessBank is accurate and efficient.

their development difficult to scale as models, tools, and application environments rapidly evolve. This motivates *agent-harness self-evolution* (Ren et al. 2026), which aims to enable an agent to diagnose its own failures and iteratively improve its harness without updating the underlying model.

Recent studies have begun to automate this process by allowing agents to inspect execution trajectories, propose harness modifications, and retain modifications that appear beneficial (Zhang et al. 2026a; Chen et al. 2026a). These studies demonstrate that harness-level adaptation can improve agent performance without model fine-tuning. However, existing approaches commonly follow a relatively greedy evolution procedure in which a small number of locally promising edits are repeatedly promoted (Lin et al. 2026; Chen et al. 2026b). Such procedures provide limited support for preserving structurally different solutions or combining complementary improvements, and may consequently collapse toward a narrow class of safe modifications, particularly prompt-level edits. Moreover, for efficiency, candidates are often selected using single-run improvements, non-regression criteria, or model-predicted utility, without explicitly verifying whether the proposed mechanism is activated or whether the observed gain exceeds execution noise (Ursekar et al. 2026; Hambardzumyan et al. 2026). These limitations make current self-evolution procedures susceptible to search collapse, task-specific overfitting, and unverifiable performance gains.

\*Corresponding author.

Based on the above observations, achieving trustworthy agent-harness self-evolution presents two fundamental challenges. **First, harness evolution requires structured exploration over a large and heterogeneous modification space.** A harness may be improved through changes to prompts, injected knowledge, runtime control logic, recovery mechanisms, tool interfaces, or numerical configurations, yielding an effectively unbounded set of possible designs. Greedy evolution tends to repeatedly exploit the current best candidate and gradually collapse toward conservative, incremental modifications after more ambitious offspring fail (Ursekar et al. 2026). It may also retain patches that memorize particular training tasks rather than correct recurring failure mechanisms. A successful self-evolution procedure must therefore preserve high-performing but semantically distinct harnesses, while enabling their useful mechanisms to be reinvented and recombined across generations. **Second, identifying genuinely improved harnesses is both difficult and computationally expensive.** Evaluating every offspring harness over the complete training set requires substantial agent rollouts, yet inexpensive evaluations on small task subsets can be unreliable. Agent outcomes fluctuate across repeated executions, particularly on borderline tasks, while sandbox crashes, verifier timeouts, and other infrastructure artifacts may be incorrectly attributed to harness modifications. Furthermore, an apparently successful patch may never activate during execution, or may overfit the tasks used to select it. Trustworthy evolution therefore requires an efficient screening protocol that distinguishes functional and reproducible improvements from inactive mechanisms, stochastic variation, and evaluation artifacts.

To address these challenges, we introduce **HarnessBank**, a trustworthy agent-harness self-evolution framework that pairs a *task agent* with a separate *evolver agent*. The task agent executes environment tasks under its current harness, whereas the evolver analyzes execution trajectories, diagnoses recurring failure mechanisms, and generates offspring harnesses without modifying the task model’s parameters. To prevent search collapse, HarnessBank introduces a *Harness Gene Bank* that preserves high-performing harnesses across semantic coordinates. Each bank cell is indexed by *where* a harness modification acts and *why* it is introduced, corresponding respectively to the modified harness component and the targeted failure pathology. Harnesses targeting different failure mechanisms remain available for subsequent evolution. The evolver can consequently produce offspring either by reinventing a mechanism from failure trajectories or by recombining compatible mechanisms preserved in different cells. Quality-biased parent selection exploits the strongest discovered lineage, while the semantic bank retains complementary solutions that would otherwise be discarded by conventional greedy evolution. To efficiently and reliably select offspring harnesses, HarnessBank further introduces *Gated Harness Screening*. Rather than immediately evaluating every offspring on the complete training set, HarnessBank first evaluates candidates on a sampled task subset and applies four sequential gates. A validity gate excludes outcomes corrupted by infrastructure failures; an activation gate verifies that the proposed modification actually executes; a

paired significance gate tests whether the observed gain is distinguishable from task-level variation; and a gain gate requires the candidate to outperform its parent. Candidates that pass the screening procedure are subsequently evaluated on the complete training set before competing for admission to the Harness Gene Bank. Across seven diverse agent benchmarks, HarnessBank consistently improves the performance of frozen task agents, with gains ranging from 5.1% to 15.4%.

Our main contributions are summarized as follows:

- We propose **HarnessBank**, an end-to-end framework for trustworthy agent-harness self-evolution. HarnessBank pairs a task agent with a separate evolver agent that iteratively diagnoses execution failures, generates candidate harnesses, and verifies their improvements without updating the underlying model parameters.
- To mitigate search collapse and task-specific overfitting, we introduce a **Harness Gene Bank** that organizes high-performing harnesses using semantic coordinates. By preserving harnesses that address distinct failure pathologies, the gene bank supports both failure-guided reinvention and cross-cell recombination, enabling structured exploration beyond greedy optimization of a single harness lineage.
- To efficiently and reliably identify improved harnesses, we propose **Gated Harness Screening**, which evaluates candidates on a sampled training subset through sequential checks of evaluation validity, mechanism activation, statistical significance, and performance gain. Only candidates that pass all gates undergo full training-set evaluation and compete for admission to the Harness Gene Bank.
- Experiments across seven diverse agent benchmarks demonstrate that HarnessBank consistently improves task agents, with performance gains ranging from 5.1% to 15.4%. Cross-model experiments further show that the evolved harnesses are model-specific rather than universally optimal, indicating that the transferable capability lies in the failure-diagnosis, structured-search, and verification process.

## 2 Related Work

### 2.1 LLM Agents and Agent Harnesses

Large Language Models (LLMs) have progressed from passive text generators to the reasoning core of autonomous agents capable of solving long-horizon tasks (Yao et al. 2023; Zhang et al. 2024; He et al. 2024). An *LLM agent* places an LLM within an iterative perception–reasoning–action loop, allowing it to maintain task state, invoke external tools, observe environmental feedback, and revise its behavior over multiple steps. This system-level perspective has motivated extensive research on tool-augmented reasoning, memory-enhanced agents, workflow construction, and adaptive control (Gao et al. 2026a; Xu, Wen, and Li 2026).

*Agent harness* denotes the executable scaffolding that surrounds a backbone LLM and converts it into a task-performing agent. It includes system and role prompts, injected knowledge and memory, tool interfaces, control-loop code, recovery and finalization policies, validators, and runtime configurations. Earlier work typically optimizes individual harness components in isolation, such as prompts, tools,

memory, or workflows (Hu et al. 2026; Su et al. 2026). More recent studies have begun to treat the harness as a unified engineering surface whose heterogeneous components jointly determine agent behavior (Gao et al. 2026a; Xu, Wen, and Li 2026). This broader view is particularly important when model weights are frozen, closed, or costly to update. However, manually developing a harness is often labor-intensive and error-prone. To address this, recent work has turned to automatic harness evolution, guided by the principle of improving agents with agents.

## 2.2 Agent-Harness Self-Evolution

A substantial body of agent-harness self-evolution can be understood as *partial agent-harness self-evolution*, where execution feedback is used to improve one restricted component. Prompt-optimization methods automatically generate, refine, or select instructions (Zhou et al. 2023; Yang et al. 2024; Khattab et al. 2024; Agrawal et al. 2026), while workflow-optimization approaches search over module compositions, operator orderings, and execution graphs (Zhang et al. 2025b; Lu et al. 2026). A parallel line updates contextual knowledge or memory from accumulated interactions (Zhang et al. 2025c; Liu et al. 2026; Yu et al. 2026; Zhang et al. 2025a). These methods establish that individual harness components can be optimized from trajectories or task scores, but their search spaces are limited.

More recent work studies *full agent-harness self-evolution*, which iteratively diagnoses execution trajectories, generates harness modifications, evaluates the resulting agent, and retains beneficial changes (Lee et al. 2026; Chen et al. 2026b). For example, Self-Harness clusters recurring failure patterns and proposes targeted harness edits under a non-regression rule (Zhang et al. 2026a). HarnessFix maps trace-grounded failure diagnoses to scoped repair operators (Chen et al. 2026a). AHE exposes harness components as editable artifacts and validates modifications against self-declared behavioral predictions (Lin et al. 2026).

Despite this progress, existing methods often rely on greedy search and noisy selection criteria, making them vulnerable to search collapse, task-specific overfitting, and unreliable gains. HarnessBank addresses these issues with a **Harness Gene Bank** for preserving semantically diverse harnesses and **Gated Harness Screening** for efficiently verifying candidate improvements.

## 3 Method

As illustrated in Figure 2, HarnessBank evolves the harness of a task agent through an iterative procedure.

### 3.1 Problem Formulation

**Agent harness and mutable surface.** Let  $M$  denote a frozen LLM and  $H$  its surrounding agent harness, including prompts, injected knowledge, tool specifications, runtime control logic, recovery mechanisms, and configurations. The resulting task agent is:

$$A_H = M \circ H. \quad (1)$$

Agent-harness self-evolution keeps the weights of  $M$  fixed and modifies only  $H$ . To preserve valid comparison, we partition the harness into an immutable kernel  $\mathcal{K}$  and a mutable surface  $\mathcal{X}$ :

$$H = \mathcal{K} \cup \mathcal{X}, \quad \mathcal{K} \cap \mathcal{X} = \emptyset. \quad (2)$$

The kernel may contain evaluation, bookkeeping, self-evolution, and interface-critical code, whereas  $\mathcal{X}$  contains the components that can be optimized.

**Optimization objective.** Let  $\mathcal{D}_{\text{tr}}$  denote the training tasks available during evolution and  $\mathcal{D}_{\text{te}}$  a test set withheld from all evolution decisions. For task  $i$ , attempt  $k$ , and harness  $H$ , let  $s_{i,k}(H) \in [0, 1]$  be the task score. We define the average utility score:

$$U(H; \mathcal{D}) = \frac{1}{|\mathcal{D}|} \sum_{i \in \mathcal{D}} \frac{1}{K} \sum_{k=1}^K s_{i,k}(H). \quad (3)$$

Harness evolution searches for:

$$H^* = \arg \max_{H \in \mathcal{H}(\mathcal{X})} U(H; \mathcal{D}_{\text{tr}}), \quad (4)$$

where  $\mathcal{H}(\mathcal{X})$  is the set of harnesses reachable through valid modifications to  $\mathcal{X}$ . The test set is used only after evolution to assess the train-selected harness.

### 3.2 Self-Evolution Based on Harness Gene Bank

HarnessBank separates four key components. The *task agent* executes tasks under the current harness. The *evolver agent* diagnoses failures and proposes patches. The deterministic *evaluator* controls sampling, scoring, activation logging, and statistical tests. The *harness gene bank* preserves high-performing harnesses across semantic cells.

Conventional greedy evolution may collapse onto a narrow class of safe edits or over-specialize patches to individual training tasks. To address this challenge, we introduce *Harness Gene Bank* (HGB), which preserves high-performing candidate harnesses for iterative reinvention and recombination. HGB is inspired by the quality-diversity principle of MAP-Elites (Mouret and Clune 2015). Unlike conventional MAP-Elites, HGB uses semantic pathology descriptors, admits only verified candidates, and prioritizes the strongest lineage under a limited rollout budget.

**Semantic cells.** At round  $t$ , HGB maintains a categorical bank

$$\mathcal{A}_t : \mathcal{W} \times \mathcal{Y} \rightarrow \mathcal{H}(\mathcal{X}) \cup \{\emptyset\}, \quad (5)$$

where  $\mathcal{H}(\mathcal{X})$  denotes the set of valid harnesses, and each cell  $c = (w, y)$  corresponds to a preserved harness. Here,  $w$  specifies *where* it acts, and  $y$  specifies *why* it is introduced. The *where* coordinate belongs to:

$$\mathcal{W} = \text{prompt, knowledge, runtime, config}, \quad (6)$$

which is determined from the modified component. The *why* coordinate belongs to an evolving pathology set  $\mathcal{Y}$  and is inferred from failure trajectories. For example, a selective recovery mechanism for reasoning-budget exhaustion occupies (*runtime, thinking-runaway*). At initialization, the gene bank is blank and  $\mathcal{A}_0(c) = \emptyset$  for any  $c$ .

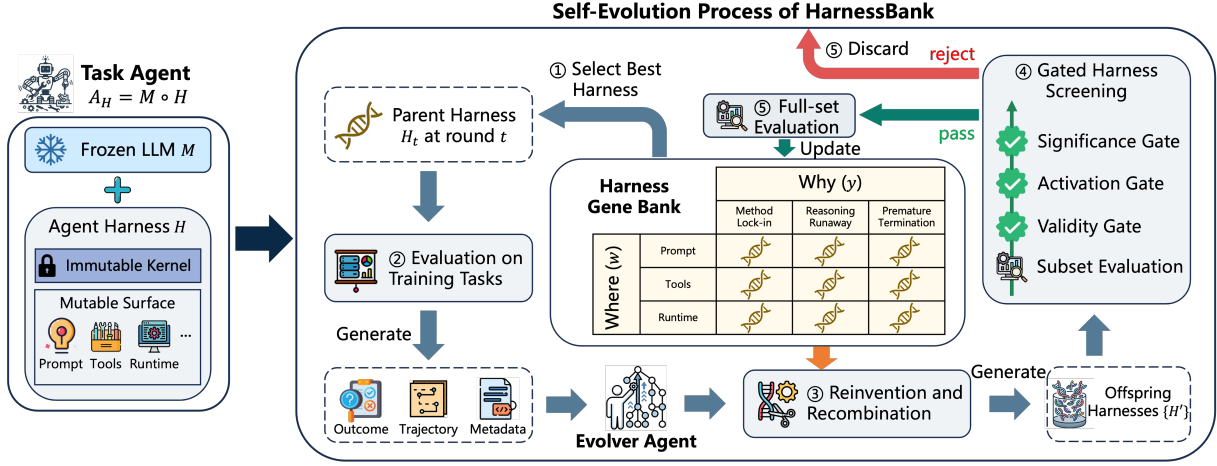


Figure 2: The framework of HarnessBank: 1) The best harness at round  $t$  is selected; 2) The comprehensive diagnosis on training tasks is obtained; 3) The offspring harnesses are reinvented or recombined based on Harness Gene Bank; 4) High-quality harnesses are filtered by Gated Harness Screening; 5) Passed harnesses are evaluated and inserted into bank cells according to the principle of competitive selection.

Harness indexes harnesses by targeted pathology rather than task-identity biases to search for reusable corrections. Harnesses addressing the same hypothesized mechanism compete within one cell and only the best one is preserved, whereas harnesses targeting different pathologies remain available for recombination. HGB preserves diverse, organized, and high-quality harnesses, which effectively expand the exploration space of offspring harness generation described in the following.

**Heredity and variation.** At round  $t$ , let  $H_t$  denote the currently best harness sampled from the vanilla harness and the harness gene bank according to the quality-biased selection rule:

$$H_t = \arg \max_{H \in (\{H_0\} \cup \text{Im}(\mathcal{A}_t)) \setminus \{\emptyset\}} U(H; \mathcal{D}_{\text{tr}}). \quad (7)$$

where  $\text{Im}(\mathcal{A}_t)$  is the image of  $\mathcal{A}_t$ , i.e., the set of all available harnesses in HGB.  $H_t$  serves as the parent harness in this round and has been evaluated on  $\mathcal{D}_{\text{tr}}$  while entering the bank. For each training task, we have its outcome  $s_{i,k}(H_t)$ , trajectory  $\tau_{i,k}(H_t)$ , and evaluation metadata  $e_{i,k}(H_t)$ , forming a comprehensive diagnosis:

$$\mathcal{L}(H_t; \mathcal{D}_{\text{tr}}) = \{(s_{i,k}(H_t), \tau_{i,k}(H_t), e_{i,k}(H_t))\}_{i \in \mathcal{D}_{\text{tr}}, k \in [K]}. \quad (8)$$

Based on the diagnosis  $\mathcal{L}(H_t; \mathcal{D}_{\text{tr}})$  and the whole HGB, the evolver agent generates a batch of offspring harnesses  $\{H'\}$  with a set  $\{(w', y')\}$  indicating *where* and *why* these harnesses are evolved. Each new harness may be either newly invented or a combination of compatible harnesses from different bank cells. For convenience, we denote the semantic descriptor  $(w', y')$  of the harness  $H'$  as  $d(H')$ .

**Evolved harness selection.** Given a batch of evolved harnesses, we need to find harnesses that actually improve upon the parent harness  $H_t$ . Since inspecting and evaluating numerous harnesses on  $\mathcal{D}_{\text{tr}}$  is costly, we propose Gated Harness

Screening to efficiently filter  $N$  harnesses  $B = \{H'_j\}_{j=1}^N$  likely to perform better than  $H_t$ , which will be introduced in detail in Section 3.3. We evaluate every filtered harness  $H'_j$  on the whole  $\mathcal{D}_{\text{tr}}$  and update its corresponding cells by the principle of competitive selection:

$$\mathcal{A}_{t+1}(d(H'_j)) = \begin{cases} H'_j, & \mathcal{A}_t(d(p)) = \emptyset, \\ H'_j, & U(H'_j; \mathcal{D}_{\text{tr}}) > U(\mathcal{A}_t(d(p)); \mathcal{D}_{\text{tr}}), \\ \mathcal{A}_t(d(p)), & \text{otherwise.} \end{cases} \quad (9)$$

Archive admission supports continued exploration but does not constitute final held-out credit.

The self-evolution of harnesses terminates after at most  $R$  rounds or after  $P$  consecutive rounds without a cell update. Finally, the best harness is selected by Eq. 7 and evaluated on the test tasks  $\mathcal{D}_{\text{te}}$ . We note that the test set is never used in the harness self-evolution.

### 3.3 Gated Harness Screening

The offspring harnesses  $\{H'\}$  at round  $t$  may contain various defects and may not improve upon the parent harness  $H_t$ . However, evaluating all of  $\{H'\}$  on the whole training set  $\mathcal{D}_{\text{tr}}$  is costly. Therefore, we propose Gated Harness Screening, which efficiently evaluates  $\{H'\}$  on a randomly sampled subset  $\mathcal{D}_{\text{sub}}$  of  $\mathcal{D}_{\text{tr}}$ , and integrates three logic gates to mitigate the subset bias and filter out defective harnesses. The computation of these gates is based on a comprehensive diagnosis on  $\mathcal{D}_{\text{sub}}$ , defined in Eq. 8.

**Validity and activation gates.** The validity gate ensures that evaluation outcomes are obtained under a functioning evaluation environment. Infrastructure failures, such as sandbox crashes or verifier timeouts, trigger a predefined repair-and-retry procedure rather than being immediately counted as agent failures. A candidate  $H'$  passes when its evaluation

produces a complete protocol-valid ledger:

$$g_{\text{valid}}(H'; \mathcal{D}_{\text{sub}}) = \mathbb{I}[\mathcal{L}(H'; \mathcal{D}_{\text{sub}}) \text{ is protocol-valid}]. \quad (10)$$

The activation gate verifies that the proposed mechanism actually executes. The increment of each candidate harness (i.e., harness patches in  $H'$  and not in  $H_0$ ) declares an activation specification and emits a deterministic beacon when triggered. Let  $b_{i,k}(H') \in \{0, 1\}$  indicate whether the increment of  $H'$  activates on task  $i$  and attempt  $k$ . Then

$$g_{\text{act}}(H'; \mathcal{D}_{\text{sub}}) = \mathbb{I}\left[\sum_{i=1}^{|\mathcal{D}_{\text{sub}}|} \sum_{k=1}^K b_{i,k}(H') > 0\right]. \quad (11)$$

An evolved but never-triggered harness is treated as inert.

**Significance gate.** HarnessBank compares candidate and parent harnesses on identity tasks. For the candidate  $H'$ , the task-level paired difference of the  $i$ th training task and total  $K$  attempts is defined as:

$$\delta_i = \frac{1}{K} \sum_{k=1}^K [s_{i,k}(H') - s_{i,k}(H_t)]. \quad (12)$$

For training tasks, the estimated improvement and paired statistic are

$$\hat{\Delta} = \frac{1}{|\mathcal{D}_{\text{sub}}|} \sum_{i=1}^{|\mathcal{D}_{\text{sub}}|} \delta_i, \quad z = \frac{\hat{\Delta}}{\hat{\sigma}_{\delta}/\sqrt{|\mathcal{D}_{\text{sub}}|}}, \quad (13)$$

where  $\hat{\sigma}_{\delta}$  is the sample standard deviation of the paired differences. The significance gate indicates whether the improvement is significant:

$$g_{\text{sig}}(H'; \mathcal{D}_{\text{sub}}) = \mathbb{I}[\hat{\Delta} > 0] \times \mathbb{I}[z \geq 1.96]. \quad (14)$$

Here, 1.96 is the 97.5th percentile of the standard normal distribution, corresponding to the conventional two-sided 5% significance level. Therefore, the paired significance gate removes between-task difficulty variance and makes the decision independently auditable.

**Harness screening.** Finally, we compute the integrated screening gate  $g_{\text{scn}}$  as follows:

$$g_{\text{scn}}(H'; \mathcal{D}_{\text{tr}}) = g_{\text{valid}}(H'; \mathcal{D}_{\text{tr}}) \times g_{\text{act}}(H'; \mathcal{D}_{\text{tr}}) \times g_{\text{sig}}(H'; \mathcal{D}_{\text{tr}}) \quad (15)$$

To improve efficiency, we sequentially compute the value  $g_{\text{scn}}$  of each candidate  $H'$  and select the first- $N$  passed harnesses. These filtered harnesses further enter the evolved harness selection for a rigorous evaluation, as introduced in Section 3.2.

## 4 Experiments

### 4.1 Experimental Setup

We evaluate HarnessBank on seven domains with disjoint train/test splits: Terminal-Bench-2 (TB2) (Merrill, Shaw et al. 2026), five from EvoAgentBench (LiveCode, Omni-MATH, BrowseComp+, GDPval, SWE-bench) (Gao et al.

2026b), and AppWorld (Trivedi et al. 2024). All experiments evolve the harness only, with the backbone frozen; by default **Qwen3.6-27B**. Claude Opus 4.8 serves as the evolver agent. The primary metric is per-task success (Pass@1) averaged over  $K=3$  attempts per task. We compare against two state-of-the-art harness self-evolution methods, GEPA (Agrawal et al. 2026) and DGM (Zhang et al. 2026b).

### 4.2 Main results

The train-selected harness improves over vanilla on *every* test set, and every gain clears the paired- $2\sigma$  bar (Table 1), from +9 to +15.4%. The recurring win is harness-level recovery of the frozen model’s dominant failure mode (empty “thinking-runaway” turns), on several domains stacked with a verify-finalize self-check (VF); our running example BrowseComp+ is one such recombination, credited at +13.9%.

The seventh domain, SWE-bench, runs the same loop on a 101/26 split: its harness is strongly credited on training (+13.9%,  $n=101$ ) and lifts the test (+5.1%), but at  $n=26$  even a real +5% effect sits below the bar ( $z=0.78$ ), so we report it as *preliminary* — small- $n$  noise, not overfitting.

### 4.3 Comparison with Existing Methods

We compare HarnessBank against two state-of-the-art methods: GEPA (Agrawal et al. 2026), which is *prompt-only*, and the Darwin Gödel Machine (DGM) (Zhang et al. 2026b), which self-modifies openly but *ungated*. Both run under HarnessBank’s protocol — same frozen Qwen3.6-27B as task agent *and* proposer, same splits, same paired- $2\sigma$  ruler — on comparable budgets (780–2,310 rollouts each, within  $2.1 \times$  on any domain), with GEPA outspending us on two domains and DGM on one (Figure 3).

Across the five sealed tests of Figure 4, HarnessBank is credited on *four* (SWE-bench falls short only at  $n=26$ ), DGM on *one*, and GEPA on *none*.

GEPA finds *no* variant beating its seed on LiveCode in 47 iterations: thinking-runaway is not prompt-addressable. Where its search returns no train-improving candidate it ships vanilla, so three of its five cells sit at the baseline. On AppWorld, whose win surface is partly prompt-expressible, its training gain washes out on the sealed test (+2.8,  $z=0.97$ ).

DGM clears the bar once: AppWorld, +10.7% ( $z=3.00$ ). Its other cells show the cost of selecting without a significance gate. On LiveCode it picks its best of 15 generations on a 15-task  $K=1$  spike (0.733, regressing to 0.533 on re-evaluation) and lands uncredited ( $z=0.66$ ); on Omni-MATH the harness it ships is *worse* than vanilla (−1.1%) — an ungated loop can deploy a regression.

The contrast is thus not search effort but what the edits can reach and what survives the gates. On SWE-bench, the one domain where a baseline scores highest, neither is credited and  $n=26$  cannot separate them.

### 4.4 Significance and Generalization

All six test gains are *credited*: each per-task paired difference clears  $z \geq 1.96$  on tasks the evolving agent never saw ( $p$  from  $< 10^{-4}$  to 0.033; AppWorld strongest at  $z=6.44$ ,  $n=168$ ).

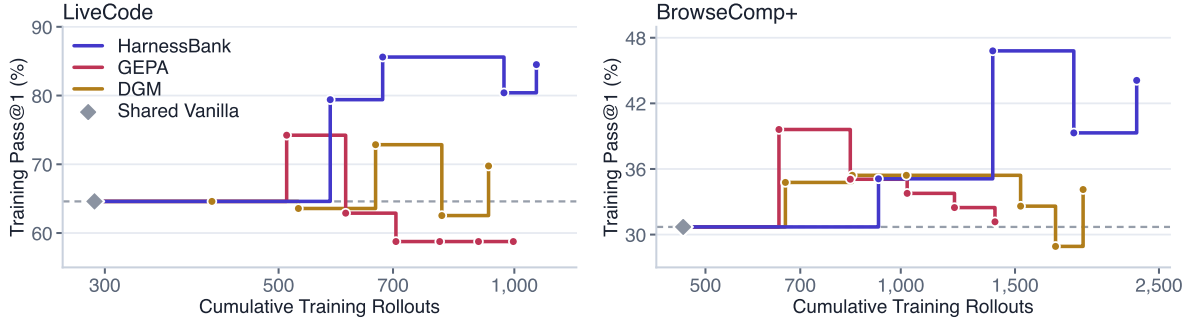


Figure 3: Evolution trajectories (frozen Qwen3.6-27B): every harness produced, in order, against cumulative rollouts (log scale), from a shared vanilla (diamond). No method climbs monotonically — GEPA’s first candidate is its best on both domains. Training scores do *not* rank methods; ranking is decided once on the sealed test (Figure 4).

Table 1: Main results of HarnessBank on seven benchmarks. *Vanilla* denotes the harness at initialization and *Evolved* denotes the self-evolved harness by HarnessBank. Parentheses = gain over vanilla (%). Ret. = test÷training gain.

| Benchmark   | Domain             | Training Pass@1 |              | Test Pass@1 |                     | Ret. | Test Pass@3 |              |
|-------------|--------------------|-----------------|--------------|-------------|---------------------|------|-------------|--------------|
|             |                    | Vanilla         | Evolved      | Vanilla     | Evolved             |      | Vanilla     | Evolved      |
| TB2         | Terminal Operation | 37.7            | 44.0 (+ 6.3) | 36.1        | <b>45.4</b> (+ 9.3) | 148% | 52.8        | 58.3 (+ 5.5) |
| LiveCode    | Code Generation    | 64.6            | 85.6 (+21.0) | 58.1        | <b>71.8</b> (+13.7) | 65%  | 66.7        | 79.5 (+12.8) |
| Omni-MATH   | Math Reasoning     | 78.4            | 91.1 (+12.7) | 54.3        | <b>66.0</b> (+11.7) | 92%  | 65.0        | 75.0 (+10.0) |
| BrowseComp+ | Web Research       | 30.7            | 46.8 (+16.1) | 16.9        | <b>30.8</b> (+13.9) | 86%  | 33.8        | 49.2 (+15.4) |
| GDPval      | Knowledge Work     | 73.6            | 82.0 (+ 8.4) | 43.7        | <b>52.9</b> (+ 9.2) | 110% | 58.6        | 67.2 (+ 8.6) |
| AppWorld    | App & API Control  | 51.9            | 69.7 (+17.8) | 41.3        | <b>56.7</b> (+15.4) | 86%  | 67.3        | 75.6 (+ 8.3) |
| SWE-bench   | Repo-Level Bug Fix | 55.4            | 69.3 (+13.9) | 47.4        | <b>52.6</b> (+ 5.1) | 37%  | 61.5        | 69.2 (+ 7.7) |

Each is a *single* comparison (the train-selected winner scored once on held-out tasks), so the many candidate comparisons during evolution cannot inflate it. The gate is two-sided: on TB2 it culled a significantly **negative** candidate (−6.4%).

Test lift retains 65–148% of train lift; none collapses. Two benchmarks exceed their training gain and three retain 86–92%; LiveCode, whose training gain is the largest in the suite, retains 65%. Retention is a ratio of two noisy estimates that a small denominator inflates (GDPval’s 110% divides by +8.4%, LiveCode’s 65% by +21.0%), so we do not treat it as a test: the load-bearing claim is the credited held-out gain.

pass@3 also rises on *every* credited domain (+5.5 to +15.4%), so the harness *expands the set of solvable tasks*, not just the per-attempt hit rate; pass@1 remains our primary metric.

#### 4.5 Diagnosis and Cost

Two pathologies recur, each fixed by a distinct RUNTIME mechanism: *thinking-runaway empty* turns, fixed by selective recovery, and *premature/unverified finalization*, fixed by a verify-finalize self-check. On five domains a single pathology dominates 49–88% of vanilla failures and the matched mechanism is credited on the test, under both deterministic and judge-based verifiers — evidence of a *true* failure mode of the frozen model rather than a scoring artifact. The two domains with heterogeneous failure maps still yield credited

and preliminary patches.

WHY is an LLM-assigned *hypothesis*, not ground truth: on AppWorld the loop misdiagnosed a capability limit as a knowledge gap, and the gate rejected the proposed patch (its own target task 0/24 → 0/24,  $p=1.0$ ). Because WHY only steers which candidates to try while credit comes solely from the gate, a wrong label costs at most a rejected candidate, never a bad harness.

#### 4.6 Cross-Model Dissociation

Thinking-runaway dominates four of six domains, but the loop is not re-fixing one qwen3.6-27B quirk. The recurring win is a *model* property (reasoning-budget overrun): the credited mechanism disables thinking only *after* a runaway, and on LiveCode neither a 16× token budget nor a blanket thinking-off toggle reproduces its gain. Cold-started on other models the loop evolves *their own* credited harnesses, each targeting that model’s distinct pathology (Table 2).

The patches follow a *pathology*→*patch matching law*: each model’s dominant pathology draws a patch that helps it and not models with a different one. On AppWorld the 27B fails by empty “engagement” turns, fixed by verify-finalize, whereas the 397B and **Gemini 3 Flash** make careless errors, fixed by a submit-verify checklist; Gemini reproduces the careless→checklist match (+13.5%,  $z=4.15$ ), extending the law across *families*. Off the diagonal the mismatched patch is near-zero on the Qwen pair (+0.2, +1.2); on Gem-



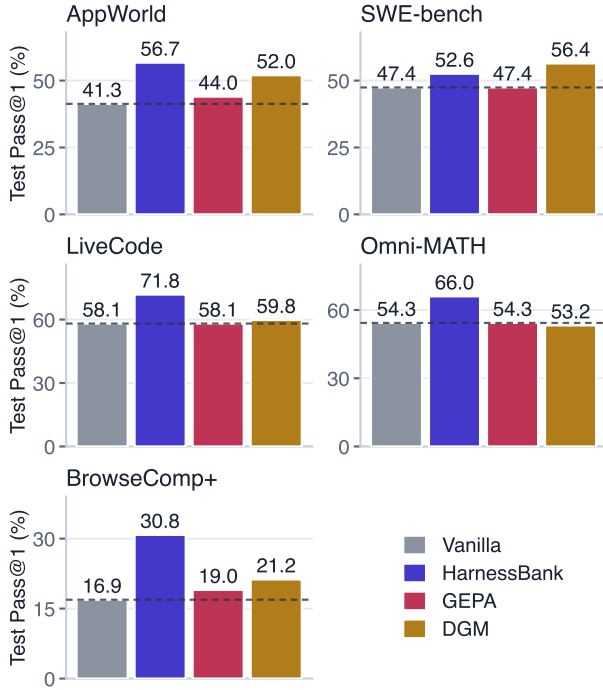


Figure 4: Test Pass@1 ( $K=3$ ) of different methods. Dashed lines denote the performance of the vanilla harness.

Table 2: Cross-model matching law: Test  $\Delta$ Pass@1 (%). **Bold** = matched patch (paired- $2\sigma$ ). A/B = VF/SV on AppWorld, 27B-stack / raise-reasoning on Omni-MATH.

| Domain    | Model  | Dominant Failure  | A             | B             |
|-----------|--------|-------------------|---------------|---------------|
| AppWorld  | 27B    | Empty Engagement  | + <b>15.4</b> | +1.2          |
|           | 397B   | Careless          | +0.2          | + <b>13.6</b> |
|           | Gemini | Careless          | +5.8          | + <b>13.5</b> |
| Omni-MATH | 27B    | Thinks Too Much   | + <b>11.7</b> | +1.7          |
|           | 397B   | Runaway (Shared)  | + <b>11.0</b> | +0.7          |
|           | Gemini | Thinks Too Little | -1.5          | + <b>15.3</b> |

ini verify-finalize still gains +5.8, a gradient rather than a switch because it shares a verify step with the checklist.

Omni-MATH supplies the converse. The two Qwen generations *share* the pathology (thinking-runaway on long competition math) and the 27B-evolved stack transfers nearly loss-free (+11.7 native, +11.0 transferred, both credited). Gemini reasons too *little* instead, so the transplanted recovery never fires (-1.5) while its matched patch turns the *same* lever the other way, to saturation (+15.3, credited). Turning it the wrong way is not merely neutral but harmful: -15.7 when stacked on the evolved 397B harness. The two families need one lever turned in opposite directions: a credited harness is a correction fitted to the model, not a universally good setting.

Table 3: Ablation results on TB2. Rows below HarnessBank denote modified variants. *False elites* = evolved harnesses whose gains in the test cannot be distinguished from noise;  $> 20$  (cap) = the stop rule is never met.

| TB2 Configuration                     | Test Pass@1 | False Elites | Rounds       |
|---------------------------------------|-------------|--------------|--------------|
| <b>HarnessBank</b> ( $K=3, 2\sigma$ ) | <b>45.4</b> | <b>0</b>     | <b>10.0</b>  |
| w/o $2\sigma$                         | $\pm 0.0$   | +2           | $> 20$ (cap) |
| w/o confirm + $2\sigma$               | -1.6        | +3           | $> 20$ (cap) |
| Vanilla                               | -9.3        | —            | —            |

## 4.7 Ablation Study

Train selection is a *lower bound* on what generalizes: on GDPval a variant ranked below the winner on train scored *highest* on test (+11.5% vs. +9.2%) — hence crediting on held-out data.

Does the paired- $2\sigma$  gate matter, or would prior work’s “mean improves” rules suffice? Table 3 ablates it on TB2 along what gets deployed, what enters the archive, and whether the loop can stop. Deployment is unchanged (train-argmax already picks VF), but two noise mechanisms enter as elites — one inert, its activation beacon never firing — and false elites then seed parents, spending future budget on noise. Termination is decisive: on post-convergence rounds with neutral candidates, phantom progress appears in 62–76% of rounds under single-run or  $K=3$ -mean crediting, so the loop stops only at the cap, whereas paired- $2\sigma$  stops at the 10-round floor.

Two signs the archive earns its keep. First, on most domains the credited harness stacks mechanisms drawn from more than one cell, and it is the per-cell elite that keeps the second mechanism alive long enough to be recombined at all. Second, accepted edits span all four levers rather than prompts alone. The diversity axis is *semantic* rather than per-task by design: an archive keyed on tasks would preserve harnesses indexed by the very tasks used to select them, which overfits by construction.

## 5 Conclusion

In this paper, we presented **HarnessBank**, a trustworthy framework for agent-harness self-evolution without updating the underlying model. HarnessBank combines a semantic Harness Gene Bank, which preserves and recombines high-performing harnesses across distinct failure pathologies, with Gated Harness Screening, which efficiently filters functional and statistically reliable improvements. Experiments across seven agent benchmarks demonstrate consistent gains of 5.1% to 15.4%. Cross-model results further show that the evolved harnesses adapt to model-specific failure patterns, suggesting that robust agent improvement arises from the diagnosis–search–verification process rather than from a universally optimal harness.

## References

Agrawal, L. A.; Tan, S.; Soylu, D.; Ziems, N.; Khare, R.; Opsahl-Ong, K.; Singhvi, A.; Shandilya, H.; Ryan, M. J.; Jiang, M.; Potts, C.; Sen, K.; Dimakis, A. G.; Stoica, I.; Klein,

- D.; Zaharia, M.; and Khattab, O. 2026. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. In *International Conference on Learning Representations (ICLR)*.
- Chen, M.; Wang, J.; Liu, Z.; Wang, Y.; and Wang, Q. 2026a. From Failed Trajectories to Reliable LLM Agents: Diagnosing and Repairing Harness Flaws. *arXiv:2606.06324*.
- Chen, T.; Lu, S.; Zhao, K.; Meng, W.; Teng, H.; Li, T.; Li, C.; Liu, X.; Liang, J.; Zhang, Z.; Xie, Y.; Qu, H.; Shao, K.; and Luan, J. 2026b. HarnessX: A Composable, Adaptive, and Evolvable Agent Harness Foundry. *arXiv:2606.14249*.
- Gao, H.-a.; Geng, J.; Hua, W.; Hu, M.; Juan, X.; et al. 2026a. A Survey of Self-Evolving Agents: What, When, How, and Where to Evolve on the Path to Artificial Super Intelligence. *Transactions on Machine Learning Research*. *ArXiv:2507.21046*.
- Gao, X.; Hu, C.; Chen, H.; Yao, P.; Wang, Z.; Bai, Y.; Wu, Z.; Han, Y.; Cong, X.; Gui, J.; Deng, Y.; and Li, T. 2026b. EvoAgentBench: Benchmarking Agent Self-Evolution via Ability Transfer. *arXiv preprint arXiv:2607.05202*.
- Hambardzumyan, K.; Baldwin, N.; Toledo, E.; Hazra, R.; Kuchnik, M.; Al Omari, B.; et al. 2026. AIRA\_2: Overcoming Bottlenecks in AI Research Agents. *arXiv preprint arXiv:2603.26499*.
- He, H.; Yao, W.; Ma, K.; Yu, W.; Dai, Y.; Zhang, H.; Lan, Z.; and Yu, D. 2024. Webvoyager: Building an end-to-end web agent with large multimodal models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 6864–6890.
- Hu, C.; Gao, X.; Zhou, Z.; Xu, D.; Bai, Y.; Li, X.; Zhang, H.; Li, T.; Zhang, C.; Bing, L.; and Deng, Y. 2026. EverMemOS: A Self-Organizing Memory Operating System for Structured Long-Horizon Reasoning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 45836–45853. Association for Computational Linguistics.
- Khattab, O.; Singhvi, A.; Maheshwari, P.; Zhang, Z.; Santhanam, K.; Vardhamanan, S.; Haq, S.; Sharma, A.; Joshi, T. T.; Moazam, H.; Miller, H.; Zaharia, M.; and Potts, C. 2024. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. In *International Conference on Learning Representations (ICLR)*.
- Lee, Y.; Nair, R.; Zhang, Q.; Lee, K.; Khattab, O.; and Finn, C. 2026. Meta-Harness: End-to-End Optimization of Model Harnesses. *arXiv:2603.28052*.
- Lin, J.; Liu, S.; Pan, C.; Lin, L.; Dou, S.; Xi, Z.; Huang, X.; Yan, H.; Han, Z.; Gui, T.; and Jiang, Y.-G. 2026. Agentic Harness Engineering: Observability-Driven Automatic Evolution of Coding-Agent Harnesses. *arXiv:2604.25850*.
- Liu, J.; Ye, X.; Xia, P.; Zheng, Z.; Xie, C.; Ding, M.; and Yao, H. 2026. EvolveMem: Self-Evolving Memory Architecture via AutoResearch for LLM Agents. *arXiv preprint arXiv:2605.13941*.
- Lu, H.; Huang, H.; Zhou, Y.; Li, C.; and Zhu, N. 2026. Empirical-MCTS: Continuous Agent Evolution via Dual-Experience Monte Carlo Tree Search. *arXiv:2602.04248*.
- Merrill, M. A.; Shaw, A. G.; et al. 2026. Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces. *arXiv preprint arXiv:2601.11868*.
- Mouret, J.-B.; and Clune, J. 2015. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*.
- Ren, Z.; Chen, Y.; Guo, D.; Rong, G.; Li, T.; Xiong, R. B.; Lan, Q.; Wang, W.; Nanbo, L.; Yang, Y.; Zhuge, M.; and Schmidhuber, J. 2026. Self-Improvements in Modern Agentic Systems: A Survey. *arXiv:2607.13104*.
- Su, J.; Fang, R.; Li, Y.; Wang, X.; Cai, S.; Xie, P.; Zhang, N.; and Yuan, F. 2026. U-Fold: Dynamic Intent-Aware Context Folding for User-Centric Agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, 18043–18057.
- Trivedi, H.; Khot, T.; Hartmann, M.; Manku, R.; Dong, V.; Li, E.; Gupta, S.; Sabharwal, A.; and Balasubramanian, N. 2024. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.
- Ursekar, V.; Shanker, A.; Chatrath, V.; Xue, Y.; and Denton, S. M. 2026. VeRO: A Harness for Agents to Optimize Agents. In *International Conference on Machine Learning (ICML)*. *ArXiv:2602.22480*.
- Xu, T.; Wen, H.; and Li, M. 2026. Adapting the Interface, Not the Model: Runtime Harness Adaptation for Deterministic LLM Agents. *arXiv:2605.22166*.
- Yang, C.; Wang, X.; Lu, Y.; Liu, H.; Le, Q. V.; Zhou, D.; and Chen, X. 2024. Large Language Models as Optimizers. In *International Conference on Learning Representations (ICLR)*.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.
- Yu, D.; Zhang, V.; Wang, H.; Chen, Y.; Hu, M.; Xu, W.; Chen, S.; Torr, P.; Yin, Z.; and King, I. 2026. Dynamic Mixture of Latent Memories for Self-Evolving Agents. *arXiv preprint arXiv:2605.21951*.
- Zhang, G.; Meng, F.; Wan, G.; Li, Z.; Wang, K.; Yin, Z.; Bai, L.; and Yan, S. 2025a. LatentEvolve: Self-Evolving Test-Time Scaling in Latent Space. *arXiv preprint arXiv:2509.24771*.
- Zhang, H.; Zhang, S.; Li, K.; Zhang, C.; Chen, Y.; Zhang, Y.; Bai, L.; and Hu, S. 2026a. Self-Harness: Harnesses That Improve Themselves. *arXiv:2606.09498*.
- Zhang, J.; Hu, S.; Lu, C.; Lange, R.; and Clune, J. 2026b. Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents. In *International Conference on Learning Representations (ICLR)*. *ArXiv:2505.22954*.
- Zhang, J.; Xiang, J.; Yu, Z.; Teng, F.; Chen, X.; Chen, J.; Zhuge, M.; Cheng, X.; Hong, S.; Wang, J.; Zheng, B.; Liu, B.; Luo, Y.; and Wu, C. 2025b. AFlow: Automating Agentic Workflow Generation. In *International Conference on Learning Representations (ICLR)*.



Zhang, K.; Li, J.; Li, G.; Shi, X.; and Jin, Z. 2024. Codeagent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 13643–13658.

Zhang, Q.; Hu, C.; Upasani, S.; et al. 2025c. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models. arXiv:2510.04618.

Zhou, Y.; Muresanu, A. I.; Han, Z.; Paster, K.; Pitis, S.; Chan, H.; and Ba, J. 2023. Large Language Models Are Human-Level Prompt Engineers. In *International Conference on Learning Representations (ICLR)*.